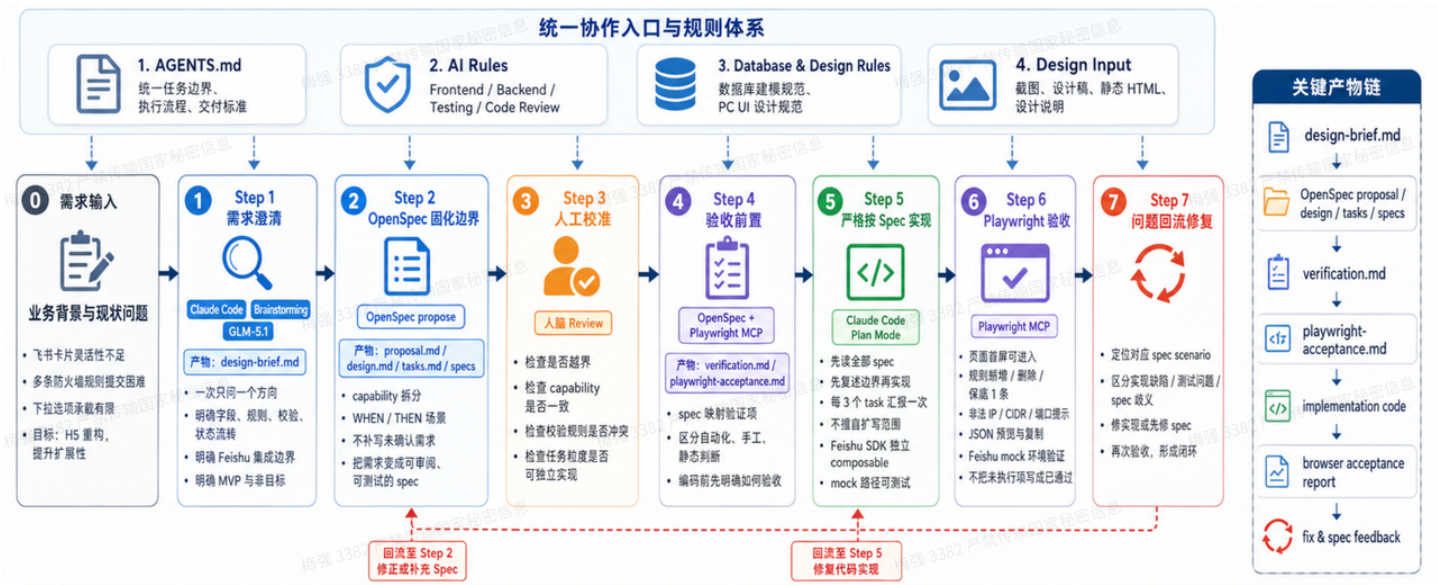


Vibe Coding 实践探索(开发示例版0521)

AI Coding 功能交付全景流程图



一、概念解释

概念	本质	解决什么问题
Rules	项目长期约束	告诉 Agent “这个项目怎么写代码,该遵守哪些规则 约束”
Skills	可复用的任务能力包	告诉 Agent “遇到某类任务时按这个流程做”，教 Agent 如何去完成某个任务的“教程”
MCP	Agent 连接外部工具的协议	让 Agent 能访问浏览器、数据库、GitHub、文档、接口等
Workflow	人和 Agent 协作的步骤	控制什么时候澄清、什么时候写代码、什么时候验收

二、实践场景

1. 安装使用的编码工具，技能&mcp

1.1 安装 AI coding IDE & code Agent (cursor、codex、claude code、trae……)

1.2 安装 superpowers

仓库地址：<https://github.com/obra/superpowers>

Superpowers 是一套完整的适用于code agent的软件开发方法论， workflow 框架。

框架中包含了多个技能

分组	技能	作用
入口/元技能	using-superpowers	整套方法论的"总开关"，教 agent 在什么场景调用哪个技能，相当于索引和调度逻辑入口/元技能 writing-skills 72.5K
规划阶段	brainstorming	写代码前做结构化的需求/方案头脑风暴，把模糊想法逼成清晰的可执行方案（ 最热门 ）
规划阶段	writing-plans	把方案落成正式开发计划文档（任务拆解、步骤、验收标准）
规划阶段	executing-plans	拿着计划一步步、可追踪地执行，避免做着做着跑偏
编码纪律	test-driven-development	强制走 TDD：先写测试再写实现
编码纪律	verification-before-completion	声称"做完了"之前必须先验证（跑测试、检查实际行为）
代码评审	requesting-code-review	教 agent 整理改动、发起一次有效的 code review
Git 与多 Agent 协作	subagent-driven-development	一个开发分支收尾的标准流程（合并、清理、收口）
Git 与多 Agent 协作	using-git-worktrees	用 git worktree 隔离不同任务的工作区，避免分支来回切换的混乱
Git 与多 Agent 协作	finishing-a-development-branch	一个开发分支收尾的标准流程（合并、清理、收口）

1.3安装 playwright-mcp

仓库地址：<https://github.com/microsoft/playwright-mcp>

Playwright 使语言学习模型 (LLM) 能够通过结构化的可访问性快照与网页进行交互，从而无需屏幕截图或视觉调整模型。它提供了一种快速、轻量级且确定性的浏览器自动化方法

1.4 安装frontend-design

命令: npx skills add <https://github.com/anthropics/skills> --skill frontend-design

创建具有独特风格、生产级品质且设计精良的前端界面。当用户要求构建 Web 组件、页面或应用程序时，可运用此技能。

1.5 安装 ui-ux-pro-max

命令: npx skills add <https://github.com/nextlevelbuilder/ui-ux-pro-max-skill> --skill ui-ux-pro-max

全面的网页和移动应用设计指南。包含 50 多种样式、161 种配色方案、57 种字体组合、161 种产品类型及其设计原则、99 条用户体验指南，以及涵盖 10 种技术栈的 25 种图表类型。

PS:也可以直接通过对话，让你的code agent帮你安装以上内容。

2. 准备规则，不要直接写代码

2.1 目的:

让模型和执行者始终基于同一套任务边界与交付标准协作。

没有 Rules 的 Agent，就像没有入职文档的新同事。

2.2 示例模版:

 **claude_docs_rules.zip**
14.04KB



目录结构:

```
1  claude_docs_rules/           #项目文件夹
2  |—— CLAUDE.md                # 主入口: 项目概览 + 规则/文档索引 (必读)
3  |—— .claude/
4      |—— rules/               # AI 长期行为规则 (按需加载)
5          |—— ai-behavior.md   # AI 行为准则: 核心原则 + 四步操作流程
6          |—— code-review.md   # 代码审查规范: 风格/逻辑/性能/安全清单
7          |—— testing-strategy.md # 测试策略: 单测/集成/E2E + 命名规范
8          |—— git-conventions.md # Git 规范: 分支、Commit Message 格式
9          |—— task-tracking.md  # 任务跟踪流程: tasks.md 断点续传机制
```

10		— error-handling.md	# 错误处理规范：编译/运行/网络/爬虫错误
分类			
11		— security.md	# 安全规范：参数校验、SQL 注入、敏感数据
12		— docs/	# 项目知识文档（给 AI 看的"项目说明书"）
13		— architecture.md	# 系统架构：技术栈 + 数据流 + 模块边界
14		— commands.md	# 命令汇总：前端/后端/爬虫常用命令
15		— scraper-strategy.md	# 爬虫策略：反爬机制 + Cookie/限流方案

```
1 # Claude Code Rules 按需加载说明
2
3 ## 背景
4
5 Claude Code 会在每次会话启动时自动加载 `.claude/rules/` 下的所有规则文件到上下文中。
6
7 规则文件越多、越长，消耗的上下文 token 就越多。无关规则加载会挤占有效上下文空间，降低
  Claude 遵守指令的可靠性。
8
9 ## paths frontmatter 机制
10
11 在规则文件顶部加 YAML frontmatter 的 `paths` 字段，可以让规则**只在 Claude 处理匹配文
  件时才加载**，而不是每次会话都加载。
12
13 ```markdown
14 ---
15 paths:
16   - "src/**/*.py"
17   - "tests/**/*.py"
18 ---
19
20 - 官方文档: https://code.claude.com/docs/zh-CN/memory#organize-rules-with-claude/rules/
```

2.3、为什么项目要加这些东西

维度	价值
可复现性	任何团队成员（或新会话）启动 AI 任务，行为可预测、可复现
可审计性	AI 的每一次代码变更都可以对照规则清单做 Review，不是"凭感觉"
降低沟通成本	把"这个项目用 Vue 3 + Composition API"这种话从对话里下沉到文件里，一次写好，所有人复用
风险前置	安全、错误处理、测试这些容易被遗漏的事，在规则层强制要求

知识沉淀	爬虫反爬、数据库连接这些"踩坑经验"以文档形式留下，而不是只存在某个人脑子里
新人友好	不仅给 AI 看，新加入的工程师也能 30 分钟看完项目全貌

⚠ 说明

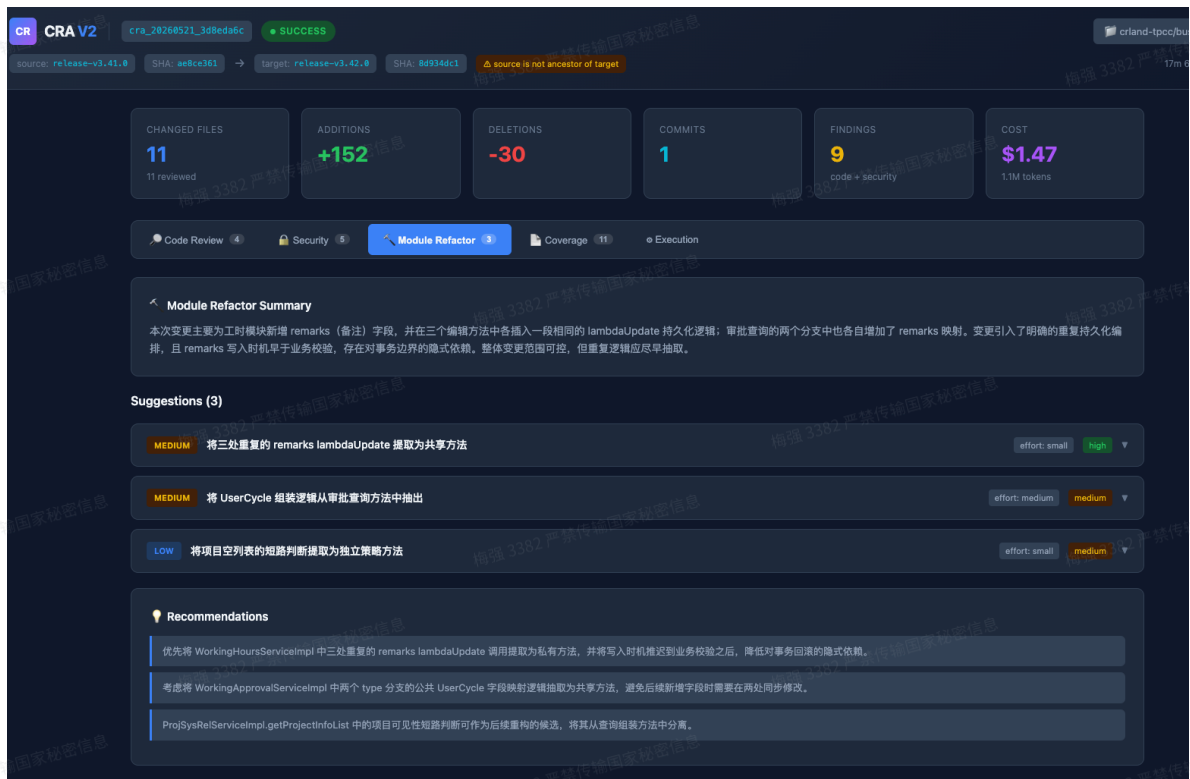
- 1.这份规则是中台项目沉淀的版本，**不要直接照抄**。
- 2.2.规则组能降低出错概率，提高交付质量，**但不能替代自动化测试与 Code Review**。
- 3.参考 Anthropic 实践：Rules 不是一次性配置，需结合实际使用持续迭代。当前版本仅为项目阶段性最优。

已有项目，生成规则文件，可使用下面的提示词

- 1 参考模板 <模板路径（**此处替换为下载的模版路径**）>，为当前项目生成规则组文件。要求：
- 2
- 3 **1.** 先完整读模板（CLAUDE.md + .claude/），再扫描当前项目（技术栈/目录/README）
- 4 **2.** 模板里的技术栈、业务描述、命令、代码示例必须全部替换成本项目的
- 5 **3.** 与本项目无关的文件直接删除，不要保留空壳
- 6 **4.** 根据本项目特性补充模板没覆盖的规则
- 7 **5.** 生成后输出**"变更说明"**：删/改/增了什么，为什么
- 8 **6.** 验证标准：新人只看这些文件能在 **30** 分钟内独立启动项目并知道哪些雷不能踩

不用工作流的效果：

- 1 根据 设计稿@docs/archive/cra-v2-design/CRA_Design_V2.md 以及
http://127.0.0.1:8000/v2/reviews/cra_20260521_3d8eda6c/result返回的结果，帮我设计
前端页面
- 2
- 3 http://127.0.0.1:8000/ui/index.html?run_id=cra_20260521_3d8eda6c



3 需求澄清，不让 Agent 猜

3.1：问题澄清，产出 设计文档 （terminal3）

目的：先把需求边界、字段定义、交互方式、技术选型和验收口径打磨清楚，再进入 spec 和实现阶段。

工具：claudecode

框架：superpowers

SKILL: brainstorming （手动调用）

(在编写代码之前启动。通过提问完善初步想法，探索各种方案，分段展示设计以供验证。保存设计文档。)

模型：GLM-5.1

输入需求：



1 `/superpowers:brainstorming` 根据 设计稿@docs/archive/cra-v2-
design/CRA_Design_V2.md 以及
http://127.0.0.1:8000/v2/reviews/cra_20260521_3d8eda6c/result返回的结果，还有pc端
ui设计规范，帮我重新设计一个前端页面，以前的保留

2

3 项目上下文探索完成。以下是关键发现：

4

5 现状：当前 `frontend/index.html` 是一个约 1090 行的单文件 SPA，使用深色主题，包含 5 个
标签页（Code Review、Security、Module Refactor、Coverage、Execution）。

6

7 API 数据：成功获取到 `cra_20260521_3d8eda6c` 的完整结果 – 4 个代码审查发现、5 个安全审
查发现、3 个模块重构建议。

8

9 核心矛盾：现有页面是深色主题，但 PC 端 UI 设计规范要求浅色中后台风格，品牌主色
#FFB000，8px 网格体系。

10

11  接下来我们会讨论页面布局、配色、组件样式等视觉相关的话题。如果你有浏览器可以打开，我可
以在后续讨论中用 mockup 展示设计方案对比，方便你直观感受效果。这个功能还比较新，会消耗较多
token。要试试吗？（需要打开一个本地 URL）

12

13 我：要

14

15  现有页面是深色主题，PC 端 UI 规范要求浅色中后台风格。新的页面你希望和旧页面放在同一个
路由下（如 `/ui`），还是放在一个新路由（如 `/ui/v2`）？

16 我：新路由

17

18

19  浏览器中现在展示了 3 种页面布局方案（http://localhost:59972）：（图在下方）

20

21 - A. Tab 切换布局 – 和现有页面结构一致，顶部 Header → 指标卡片 → Tab 标签页

22 - B. 三栏并列布局 – 代码审查/安全审查/模块重构三栏同时展示

23 - C. 左侧导航 + 右侧内容 – 左侧固定导航栏，右侧展示选中模块详情

24

25 点击卡片可以选中。看看觉得哪种更合适？

26

27

CRA V2 审查报告 — 页面布局方案

选择你偏好的整体页面结构



1 我：C

2

3 左侧导航栏你希望分几个主要区域？

4

5 1. 6 个完整分区 (Recommended)

6 总览、代码审查、安全审查、模块重构、覆盖率、执行详情 — 每个 API 数据段对应一个导航项。

7 2. 5 个分区 (覆盖率合并)

8 总览、代码审查、安全审查、模块重构、执行详情 — 覆盖率合并到总览页面下方。

9 3. 3 个大分区

10 总览、审查结果 (代码+安全+重构合一)、执行详情 — 信息更精简。

11 4. Type something.

12

13 我：5

14

15 浏览器已更新，展示了 Finding 卡片的两种交互方案 (<http://localhost:59972>)：(图在下方)

16

17 - A. 列表卡片 + 展开/收起 - Finding 以卡片列表排列，点击展开显示描述、场景、建议等详情

18 - B. 左侧列表 + 右侧详情面板 - 紧凑列表在左，选中 finding 的详情在右面板

19

20 看看哪种更适合？

21

22

Finding 卡片交互设计

选择审查发现 (Finding) 的展示与交互方式



1 我：A

2

3 1. 单文件 SPA（推荐）

4 和现有页面一样，所有 CSS/JS/HTML 内联在一个 index.html 文件中，无构建工具、无外部依赖。直接挂载到 /ui/v2 即可。

5 2. 多文件原生 JS

6 HTML、CSS、JS 拆分为独立文件，放在 frontend/v2/ 目录下。更易于维护，但仍无框架和构建工具。

7 3. 前端框架

8 使用 Vue/React + 构建工具，组件化开发。需要引入 node 构建流程。

9 4. Type something.

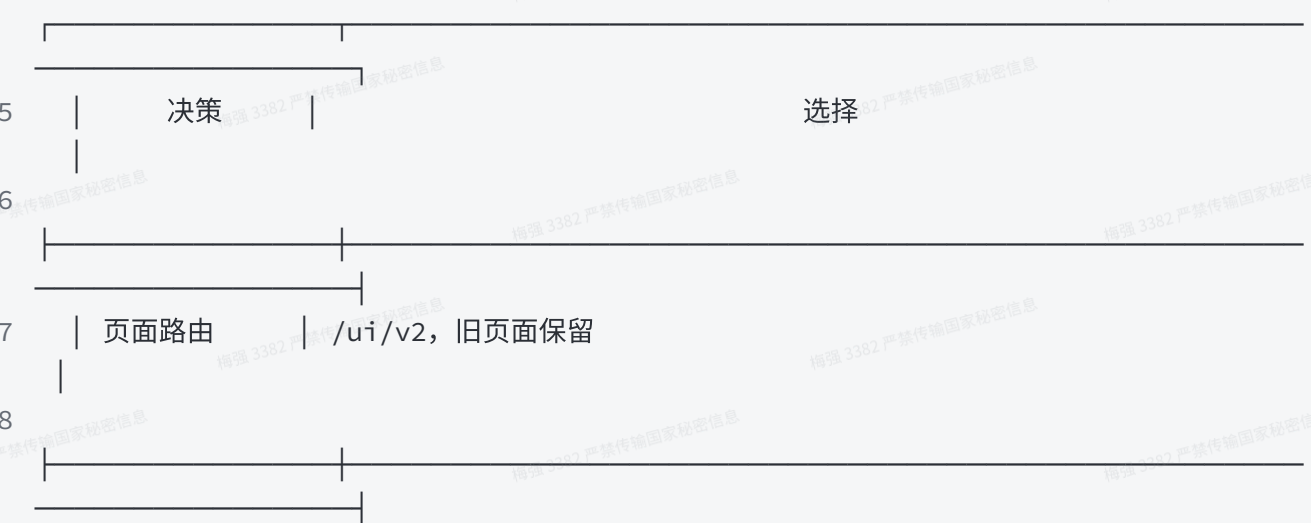
10

11 我：1

12 好，所有关键决策已确认：

13

14



19 | 整体布局 | 左侧导航 + 右侧内容

20

21 | 导航分区 | 5 个：总览（含覆盖率）、代码审查、安全审查、模块重构、执行详情 |

22

23 | Finding 卡片 | 列表卡片 + 展开/收起

24

25 | 技术实现 | 单文件 SPA（内联 CSS/JS/HTML）

26

27  现在开始呈现详细设计。我先用浏览器展示完整的页面 mockup，然后逐段确认。

28

29

30 浏览器中展示了 总览页面 的完整 mockup (<http://localhost:59972>)，包含：（图在下方）

31

32 - 顶部 Header（品牌 Logo + 仓库链接 + 状态 badge + ref 信息）

33 - 左侧导航（5 个分区，带 finding 计数徽章）

34 - 右侧内容区：5 个指标卡片、审查总结、严重程度分布柱状图、覆盖率环形图 + 文件清单预览、警告条

35

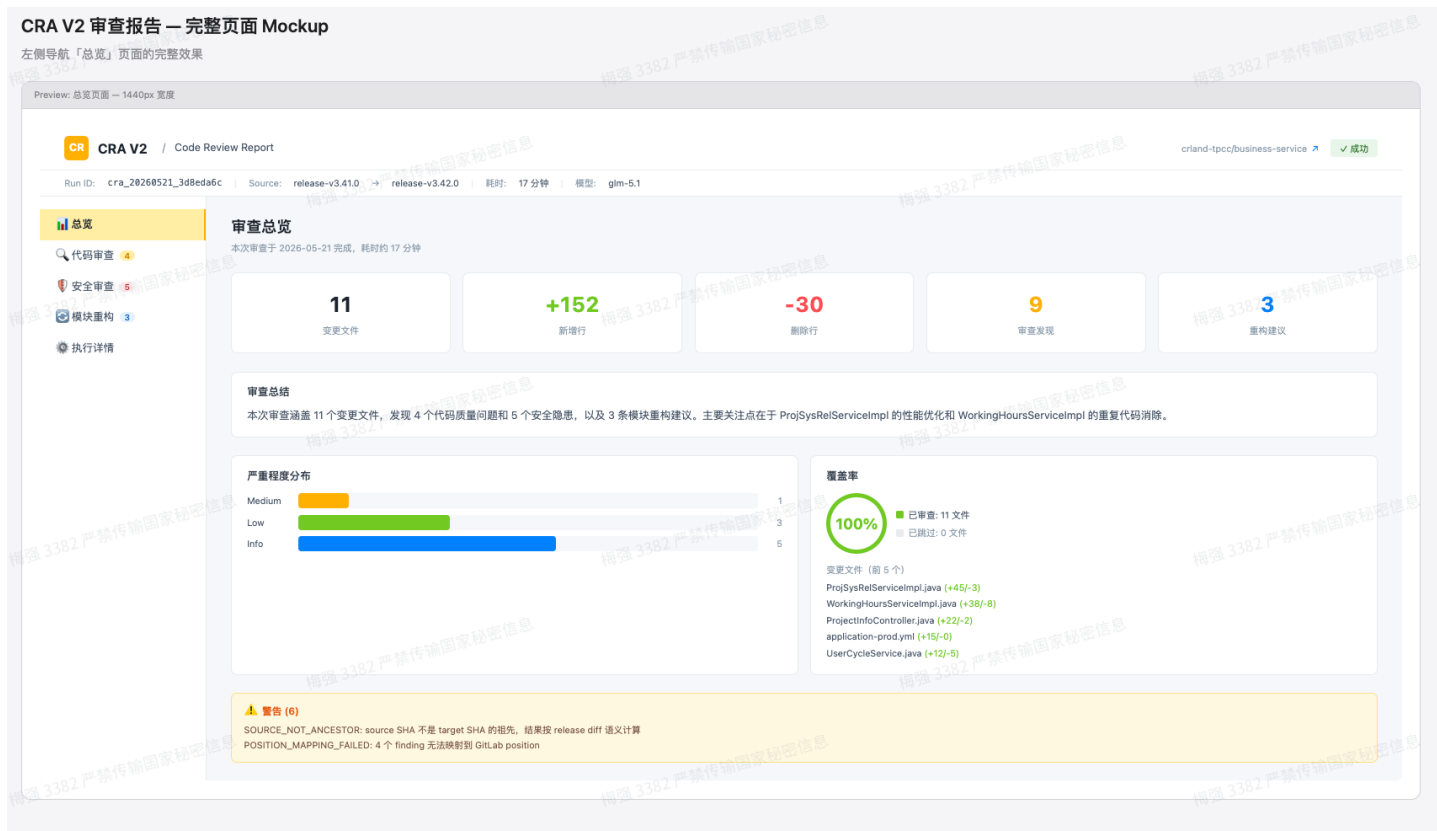
36 先看总览这部分，布局和信息展示你觉得合适吗？有没有想调整的地方？

37

38 我：没问题

CRA V2 审查报告 — 完整页面 Mockup

左侧导航「总览」页面的完整效果



1 浏览器已更新为 代码审查页面 (<http://localhost:59972>) :

2

3 - 顶部: summary 摘要卡片 + severity 过滤栏 (全部/Medium/Low/Info)

4 - Finding 卡片列表: severity 徽章 + category 标签 + 置信度 + 标题 + 文件位置

5 - 展开状态: 描述、场景 (双栏)、建议 (绿色高亮)、证据 (代码片段, 深色背景)

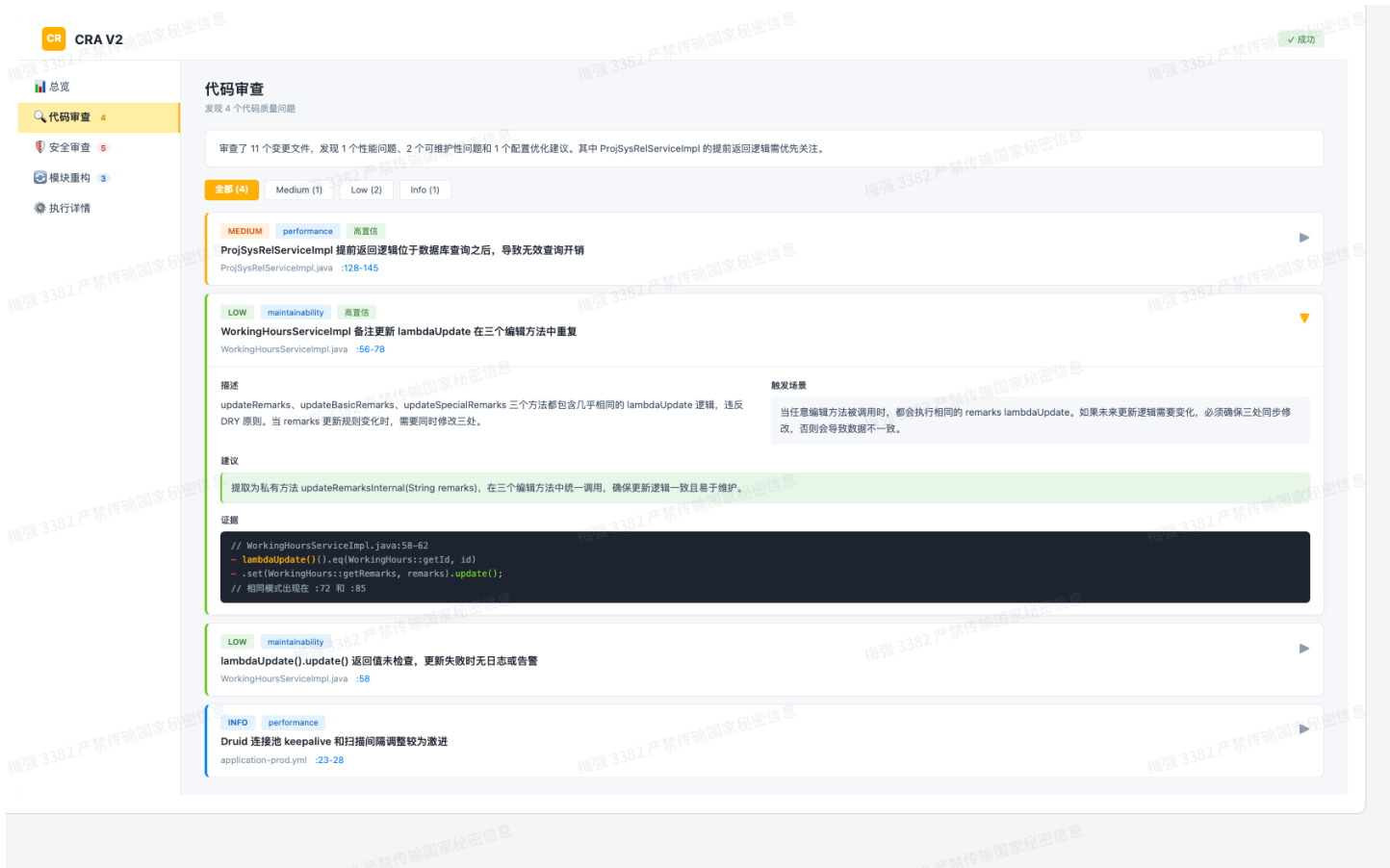
6 - 左侧 border 颜色按 severity 区分 (Medium=黄、Low=绿、Info=蓝)

7

8 代码审查页面的设计和展示效果你觉得如何?

9

10 我: 很好



- 1 安全审查页面你希望展示哪些类型的 finding? 当前 API 返回了 4 类安全数据。
- 2
- 3 1. [✓] LLM Findings
- 4 LLM 发现的 5 个安全 finding, 结构和代码审查类似 (severity/category/scenario 等)
- 5 2. [✓] Secret Findings
- 6 secret scanner 发现的敏感信息 (规则 ID、文件、行号、redaction marker)
- 7 3. [✓] SAST Findings
- 8 外部 SAST 工具的安全发现 (tool/rule_id/raw_severity/scope_status)
- 9 4. [✓] Merged Findings
- 10 合并后的最终安全发现 (source=llm+sast 时表示 LLM 和 SAST 交叉验证)
- 11 5. [] Type something
- 12 Submit

16 浏览器展示了两个页面 (<http://localhost:59972>) :

18 安全审查页面:

- 19 - 二级 Tab 切换: Merged / LLM / Secret / SAST 四类 finding
- 20 - Merged 为默认视图, 卡片增加「来源」标签 (LLM / LLM+SAST)
- 21 - 增加「triage_status」标签 (confirmed / potential 等)
- 22 - Finding 展开逻辑和代码审查一致

24 执行详情页面:

- 25 - 双栏卡片: 运行信息 + 模型使用

26 - Phase 耗时横向柱状图, LLM 阶段用品牌色高亮, 其他阶段用蓝色

27

28 这两个页面觉得可以吗?

29

30

31 我: 可以

32

33 Loading、Error、Empty、审查失败 (status=failed) 等边界场景你希望支持哪些?

34

35 1. 全部支持 (推荐)

36 加载中显示骨架屏, 失败显示错误提示, 无数据显示空状态引导, 审查失败显示错误详情页。

37 2. 仅加载和错误

38 只做加载和错误状态, 空状态用简单文案即可。

39 3. 仅加载状态

40 最小化: 只保留加载中状态, 其余复用现有逻辑。

41 4. Type something.

42 我: 全部支持

43

44 ---结尾 (图在下方) ----

45

46 设计文档已通过审查并提交到 docs/superpowers/specs/2026-05-22-cra-v2-report-page-redesign.md。

47 请查看这份设计文档, 确认没有问题后我就开始创建实施计划。需要修改的地方告诉我。

48

Terminal Local x Local (2) x Local (3) x + v

```
1 <div style="display:flex;align-items:center;justify-content:center;min-height:60vh">
2   <p class="subtitle">Continuing in terminal...</p>
3 </div>
```

● 设计文档已通过审查并提交到 docs/superpowers/specs/2026-05-22-cra-v2-report-page-redesign.md。

请查看这份设计文档, 确认没有问题后我就开始创建实施计划。需要修改的地方告诉我。

* Worked for 2m 20s

9 tasks (7 done, 1 in progress, 1 open)

- 用户审查设计文档
- 过渡到实施规划
- ✓ 探索项目上下文
- ✓ 提供视觉伴侣选项
- ✓ 提出 2-3 种设计方案
- ... +4 completed

) |

▶ accept edits on (shift+tab to cycle) : ctrl+it to hide tasks

4 大模型 或者人工 重新review 设计文档 (强烈推荐、手动调用、terminal 2新终端/会话、多轮)

```
Terminal Local x Local (2) x Local (3) x + v
(cra) daiwenxiong@daiwenxiongdeMacBook-Pro ai-review-crland % claude
Claude Code v2.1.143
glm-5.1 · API Usage Billing
~/BEAR_FILE/PROJECT/AI-PROJECT/ai-review-crland

↑ Install the PyCharm plugin from the JetBrains Marketplace: https://docs.claude.com/s/claude-code-jetbrains

) 设计文档已通过审查并提交到 docs/superpowers/specs/2026-05-22-cra-v2-report-page-redesign.md。 你帮我review一下看有没有问题
```

```
1 设计文档已通过审查并提交到 docs/superpowers/specs/2026-05-22-cra-v2-report-page-
redesign.md。 你帮我review一下看有没有问题
2
3
4 我来读取设计文档进行审查。
5
6 Read 1 file (ctrl+o to expand)
7
8 让我检查一下相关的 schema 和现有前端，验证设计文档与实际数据结构是否一致。
9 Explore(Check API schema and frontend)
10 | Read(frontend/index.html)
11 | Read(frontend/index.html)
12 | Read(frontend/index.html)
13 ... +9 tool uses (ctrl+o to expand)
14 .....
15
16
17 Review 结果
18
19 阻塞性问题
20
21 1. Severity 过滤与分布图缺少 Critical/High 级别 (4.1 & 4.2)
22
23 Schema 的 severity 枚举包含 critical, high, medium, low, info 五级，设计文档 6.4
节也定义了全部五级的颜色映射。但：
24 - 4.1 总览页的严重程度分布图只展示了 "Medium / Low / Info" 三级
25 - 4.2 代码审查页的过滤按钮组只有 "全部/Medium/Low/Info"
26
27 Critical 和 High 级别的 finding 会被完全忽略，无法被过滤或统计。建议统一为五级，或说
明简化理由。
28
29 2. out_of_scope_sast_findings 未纳入设计 (4.3)
30
31 Schema security_review_result 有 7 个 required 字段，其中
out_of_scope_sast_findings 是独立的 required 数组。设计文档 4.3 节的安全审查二级
Tab 只列了 Merged / LLM / Secret / SAST 四个 Tab，没有处理
32 out_of_scope_sast_findings。
33
```


建议增加展示方案，比如在 SAST Tab 内分区展示 in_scope / out_of_scope，或新增独立 Tab / 折叠区。

2. out_of_scope_sast_findings 未纳入设计 (4.3)

Schema security_review_result 有 7 个 required 字段，其中 out_of_scope_sast_findings 是独立的 required 数组。设计文档 4.3 节的安全审查二级 Tab 只列了 Merged / LLM / Secret / SAST 四个 Tab，没有处理 out_of_scope_sast_findings。

建议增加展示方案，比如在 SAST Tab 内分区展示 in_scope / out_of_scope，或新增独立 Tab / 折叠区。

3. 路由挂载可能冲突 (8.2)

现有 http_server.py:473 已将 frontend/ 目录挂载到 /ui：
app.mount("/ui", StaticFiles(directory=str(frontend_dir), html=True),
name="frontend")

设计文档建议新增：
app.mount("/ui/v2", StaticFiles(...), name="frontend-v2")

Starlette 按注册顺序匹配 mount，/ui 的 catch-all 会先拦截 /ui/v2/ 路径。有两种解法：

- 方案 A：不新增 mount，直接将 frontend/v2/index.html 放在现有 frontend/ 目录下，现有 mount 自动在 /ui/v2/ 路径提供服务
- 方案 B：新增 mount 但必须注册在 /ui mount 之前

设计文档需要明确选择哪种方案。

一般性问题

4. Finding 卡片缺少字段映射

Schema 的 review_finding 有 15 个 required 字段，设计文档未明确以下字段的展示方式：

字段	问题
id	未提及是否展示（可用于锚点定位）

67		
68	related_finding_ids	未提及如何展示关联关系
69		
70	gitlab_position	1.2 节明确说要展示，但详细设计中无对应实现
71		
72	confidence	4.2 节写"（如有）"，但 llm_review_finding 中 confidence 是 required 字段，应始终展示
73		
74	category	有 badge 但未定义各 category 的标签文案和颜色
75		
76	---	
77	总结	
78		
79	设计整体结构清晰，模块划分合理，视觉规范与 PC 端设计规范一致。主要问题集中在：schema 字段覆盖不完整（out_of_scope_sast_findings、gitlab_position、sast 独有字段）、severity 级别不一致（五级定义 vs	
80	三级过滤/展示）、以及路由挂载冲突。建议先解决三个阻塞问题再进入实现。	
81		

把上面审查出来的问题再喂给（terminal3）

TerminalLocal × Local (2) × Local (3) × + ▾

- 设计文档已通过审查并提交到 docs/superpowers/specs/2026-05-22-cra-v2-report-page-redesign.md。

请查看这份设计文档，确认没有问题后我就开始创建实施计划。需要修改的地方告诉我。

* Worked for 2m 20s

有些问题：

- 请说，我记录下来一起调整。

* Baked for 5s

9 tasks (7 done, 1 in progress, 1 open)

- 用户审查设计文档
- 过渡到实施规划
- ✓ 探索项目上下文
- ✓ 提供视觉伴侣选项
- ✓ 提出 2-3 种设计方案
- ... +4 completed

) [Pasted text #1 +99 lines]

1	所有问题已修复并提交。修改汇总：		
2			
3			
4	问题		修复
5			
6	1. Severity 缺 Critical/High		过滤栏和分布图统一为五级
7			
8	2. out_of_scope_sast_findings		安全审查增加第 5 个 Tab
9			
10	3. 路由挂载冲突		改为复用现有 /ui 挂载，不新增 mount
11			
12	4. Finding 字段映射缺失		补充 id、related_finding_ids、gitlab_position 链接、confidence 始终展示
13			

14	5. SAST 字段不完整	补充 message、raw 字段
15		
16	6. model_usage 不具体	按 key 分行展示 tokens/turns/calls, 汇总合计
17		
18	7. Header ref 不明确	展示 value + SHA tooltip + web_url fallback
19		
20	8. Coverage 过于简略	补充 status/review_eligible/skip_reason/partial_context + skipped_files
21		
22	9. partial_success 未处理	新增处理方案 (黄色 badge + errors 红色条)
23		
24	10. 分布图统计口径 merged)	明确 code findings + llm_findings (非 merged)
25		

26
27 请再看看修改后的设计文档是否还有问题。

28
29 我：没问题



 2026-05-22-cra-v2-report-page-redesign.md

5 任务拆解（superpowers自动调用、terminal3）

目的： 步骤4最终生成的设计文稿（2026-05-22-cra-v2-report-page-redesign.md）拆分成可执行的任务

工具： Claude Code

SKILL： writing-plans

（固化本次需求与任务边界,把为什么要改、准备改什么、边界到哪里、如何验证）

模型： GLM-5.1

```
1 好，进入实施规划阶段。调用 writing-plans skill 创建实施计划。（图在下方）
2
3  Skill(writing-plans)
4   | Successfully loaded skill
5
6  我正在使用 writing-plans 技能来创建实施计划。
7
8   我先阅读 spec 并了解当前的代码库结构。
9
10
```

TerminalLocal × Local (2) × Local (3) × + ×

```
done

• 好，进入实施规划阶段。调用 writing-plans skill 创建实施计划。

• Skill(writing-plans)
  | Successfully loaded skill

• 我正在使用 writing-plans 技能来创建实施计划。

我先阅读 spec 并了解当前的代码库结构。

Read 1 file (ctrl+o to expand)

• Explore(Explore existing frontend and server)
  | Done (7 tool uses · 22.7k tokens · 31s)
  (ctrl+o to expand)

• 现在我已掌握全部上下文。让我来编写实现计划。

• Bash(mkdir -p /Users/daiwenxiong/BEAR_FILE/PROJECT/AI-PROJECT/ai-review-crland/docs/superpowers/plans)
  | Done

* 过渡到实施规划... (1m 18s · ↓ 2.1k tokens · thought for 7s)
  | ■ 过渡到实施规划
  | ✓ 用户审查设计文档
  | ✓ 探索项目上下文
  | ✓ 提供视觉伴侣选项
  | ✓ 提出 2-3 种设计方案
  | ... +4 completed
```

1 实施计划已完成并保存到 docs/superpowers/plans/2026-05-22-cra-v2-report-page-redesign.md。共 9 个 Task，按顺序：

2

3

4	Task	内容	产物
5			
6	1	创建目录 + HTML 骨架 + CSS 变量	frontend/v2/index.html
7			
8	2	JS 基础架构 (fetch/状态/导航/骨架屏)	<script> 核心框架
9			
10	3	Header 渲染	仓库链接、状态 Badge、Ref 信息
11			

12	4	总览页	指标卡片、严重程度分布、覆盖率
13			
14	5	代码审查页	Finding 卡片、展开/收起、五级过滤
15			
16	6	安全审查页	5 个子 Tab、SAST/Secret 专用展示
17			
18	7	模块重构 + 执行详情	Suggestion 卡片、Phase 耗时图
19			
20	8	边界场景	失败页、空状态
21			
22	9	最终集成验证	端到端检查
23			
24	两种执行方式：		
25			
26	1. Subagent-Driven（推荐） – 每个 Task 派遣独立子代理执行，任务间审查，快速迭代		
27			
28	2. Inline Execution – 在当前会话中逐步执行，带检查点		
29			
30	选择哪种方式？		
31			
32	此时先不着急执行，看步骤6先review一遍		

6 大模型 或者人工 重新review 计划文档（强烈推荐、手动调用、terminal 2 新终端/会话、多轮）

1 实施计划已完成并保存到 docs/superpowers/plans/2026-05-22-cra-v2-report-page-redesign.md。帮我review一下看有哪些问题？（图在下方）

2

```
Terminal Local x Local (2) x Local (3) x + v
| Done (7 tool uses · 35.0k tokens · 46s)
(ctrl+o to expand)

• 现在我来检查计划中提到的具体 schema 字段，以验证其正确性。

Read 1 file (ctrl+o to expand)

• 对 spec 和 API schema 的交叉验证完成，以下是我发现的问题：

---
Review 结果

P0 - 阻塞性问题

1. XSS 风险：API 数据直接插入 innerHTML

整个计划通过模板字符串将 API 返回的 finding.title、finding.description、finding.evidence、finding.recommendation 等字段直接拼入 innerHTML。evidence 字段经常包含代码片段（含
<、>、&），极可能被浏览器解析为 HTML 标签甚至 <script>。

计划需要增加一个 HTML 转义函数，所有 API 数据在插入 DOM 前必须经过转义：

function esc(s) {
  if (s == null) return '';
  return String(s).replace(/&g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;').replace(/"/g,'&quot;');
}

这影响了 Task 4-8 的所有渲染函数。特别是 evidence 的 <pre> 区域和 description/recommendation 区域。

2. SAST finding 与 renderFindingCard 结构不兼容

renderFindingCard (Task 5) 在展开态渲染 finding.description 和 finding.scenario，但 sast_finding schema 中：
- 没有 description 字段 - 有 message 代替
```

把上面审查出来的问题再喂给（terminal3）

1 我觉得计划文件还有需要修改的地方，先解决，再往下执行：[Pasted text #1 +111 lines]

```
.gitlab-ci.yml
M.I. AGENTS.md

Terminal Local x Local (2) x Local (3) x

9 最终集成验证 端到端检查

两种执行方式：

1. Subagent-Driven（推荐） - 每个 Task 派遣独立子代理执行，任务间审查，快速迭代
2. Inline Execution - 在当前会话中逐步执行，带检查点

选择哪种方式？

* Sautéed for 4m 20s

) 我觉得计划文件还有需要修改的地方，先解决，再往下执行：[Pasted text #1 +111 lines]
```



 2026-05-22-cra-v2-report-page-redesign.md

7 任务执行（superpowers自动调用、terminal3）

工具：Claude Code

SKILL：subagent-driven-development

1 好，计划已就绪。两种执行方式：

2

3 1. Subagent-Driven（推荐） – 每个 Task 派遣独立子代理，任务间审查

4 2. Inline Execution – 当前会话逐步执行

5

6 选哪种？

7

8 我：1

7.1 为什么建议用子代理开发模式

单agent执行遇到的问题

问题一：上下文太长，决策质量下降

所有信息都塞给一个 Agent 时，它不一定真的会正确使用。

Subagent 的好处是每一步都重新收敛上下文，且上下文独立

问题二：没有独立审查，错误更容易漏掉

如果一个 Agent 同时负责：写代码 + 写测试 + 审查代码 + 宣布完成，很容易产生“自我确认”。

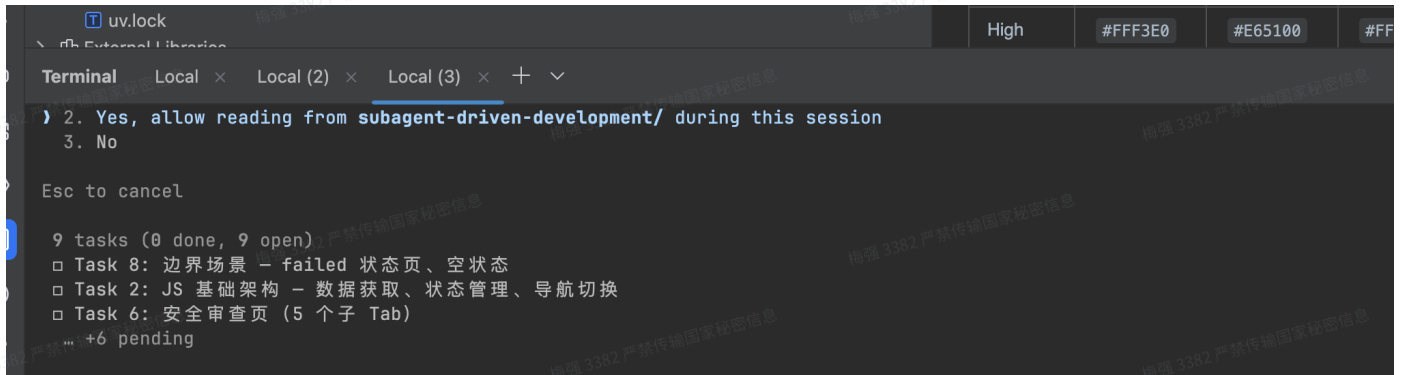
Subagent 独立 Verification Agent 或 Reviewer Agent 可以专门检查这些问题。

问题三：没有独立审查，错误更容易漏掉

如果单 Agent 拥有所有工具，它可能做出危险操作：直接改数据库、直接删除文件、直接修改配置……

Subagent 可以把风险拆开

- 1 Database Agent：只读 schema，不允许写数据。
- 2 Backend Agent：可以改代码，但不能访问生产数据库。
- 3 Reviewer Agent：只能看 diff，不能改代码。
- 4 Playwright Agent：只能操作测试环境浏览器。

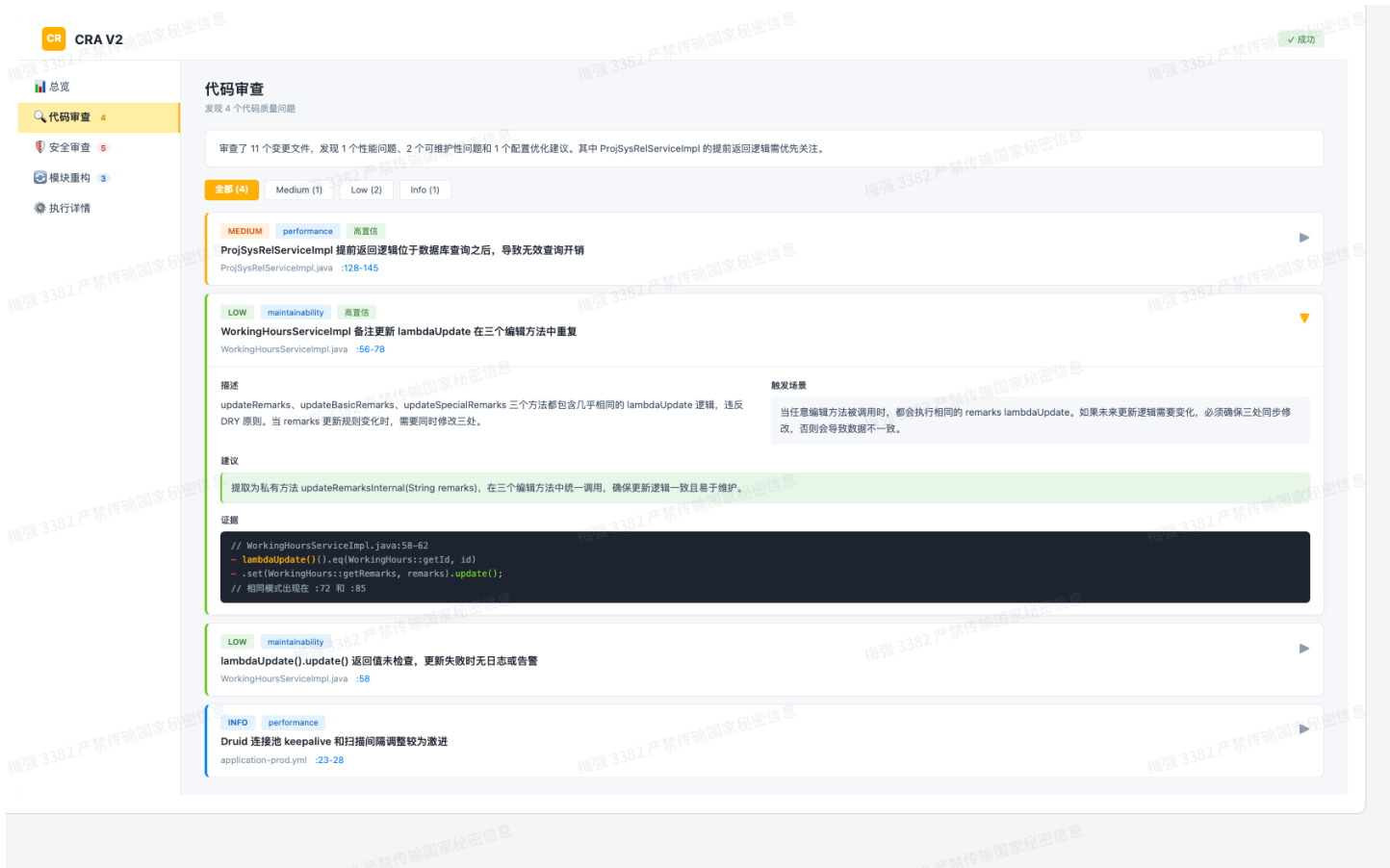


```
uv.lock
High #FFF3E0 #E65100 #FF
Terminal Local x Local (2) x Local (3) x + v
2. Yes, allow reading from subagent-driven-development/ during this session
3. No
Esc to cancel
9 tasks (0 done, 9 open)
  Task 8: 边界场景 - failed 状态页、空状态
  Task 2: JS 基础架构 - 数据获取、状态管理、导航切换
  Task 6: 安全审查页 (5 个子 Tab)
  ... +6 pending
```



```
* Compacting conversation... (11m 30s · ↑ 25.5k tokens)
===== 32%
  Task 8: 边界场景 - failed 状态页、空状态
  Task 6: 安全审查页 (5 个子 Tab)
  Task 7: 模块重构页 + 执行详情页
  Task 9: 最终集成验证
  ✓ Task 2: JS 基础架构 - 数据获取、状态管理、导航切换
  ... +4 completed
  ▶ accept edits on (shift+tab to cycle) · esc to interrupt · ctrl+t to hide tasks
```

执行后，效果图：



8 端到端浏览器级验收（Playwright MCP）

目的：在代码实现完成后，对关键用户路径做真实浏览器验证，并输出事实化验收结论。

工具：Claude Code

框架：Playwright MCP

场景二 存量系统改造

产品力系统（天窗性能工具）从 C/S 架构改为B/S架构

天窗性能工具

功能选择：

天窗性能参数推荐

既有天窗改造节能计算

天窗类型：

圆形

窗地比：

0.2

光热气候区：

严寒&4级

标准日照小时数（5~9月,10am~17pm）：1224

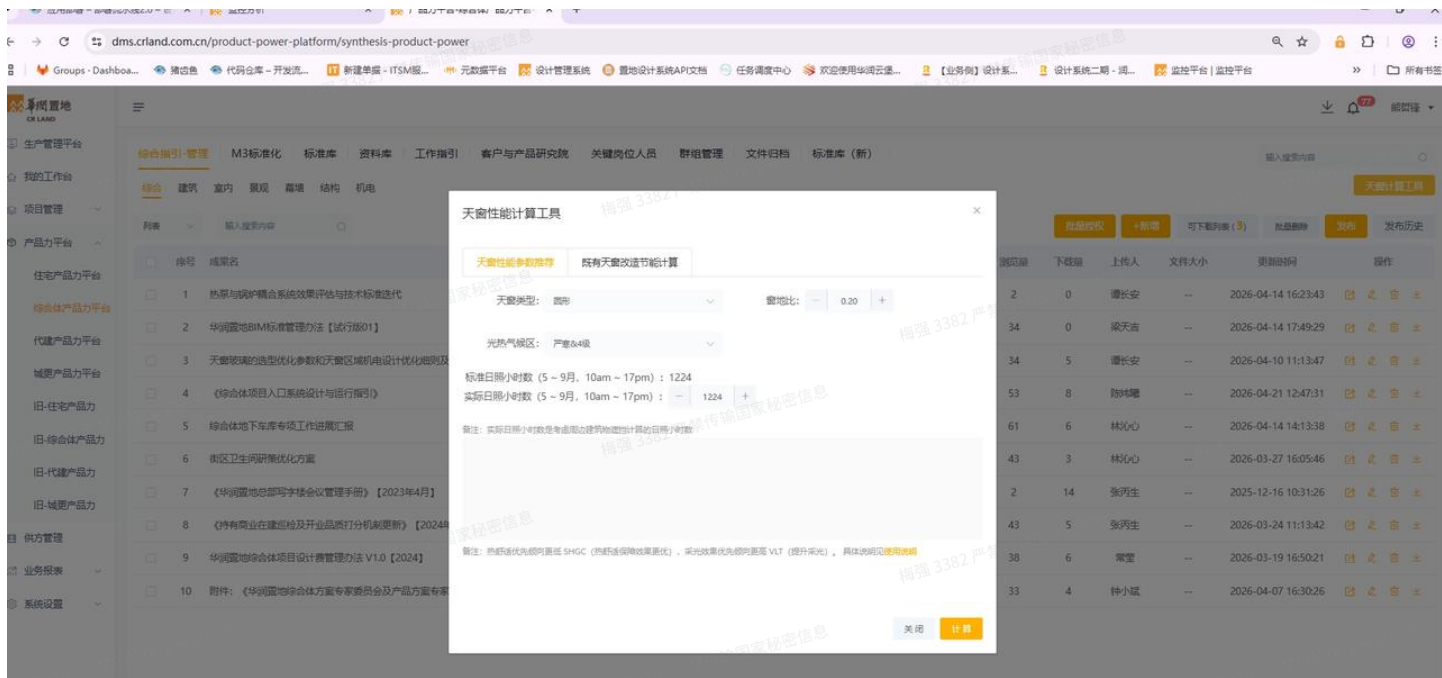
实际日照小时数(5~9月,10am~17pm)：

1224.0

备注：实际日照小时数是考虑周边建筑物遮挡计算的日照小时数

计算

© 2025 华润置地-深圳清华科技创新联合研究院 可持续发展联合研发中心. All rights reserved.



大模型更擅长的任务类型

在此类任务中合理借助大模型的能力，能够显著提升开发效率与交付质量。

- 目标态已经被明确表达** 源代码、源文档、参考样板里已经包含了"答案应该长什么样"的全部信息，模型只需要做转换，而不是创造。架构迁移、语言转换、格式改写都属于这类。
- 有强对照、可验证** 存在一个"标准答案"或基准，且可以逐段比对的任务。
- 模式在训练语料中高频出现** Java→Python、Spring→FastAPI、B/S→C/S 这类转换，GitHub 和技术社区里有海量对照样本。模型见过的同类案例越多，发挥越稳定。

反过来，模型不擅长的就是高不确定的任务：需求模糊、无对照基准、训练语料里没有同构案例、要求全局一致性、决策密度高——典型代表就是"在已有项目里从零设计一个新功能"。（所以为了提高交付质量，我们才需要在场景中一加入大量的约束）

场景三 现有产品能力扩展类、开源项目二开

示例：SQL审核平台支持达梦数据库

工具：ChatGPT、claude code

模型：GPT5.4、GLM5

3.1 背景：本次实践以开源 SQL 审核平台 Archery 为改造对象，目标是在现有架构基础上进行二次开发，新增对国产数据库 达梦数据库（DM / Dameng）的支持能力。

该类任务不同于普通业务页面开发，具有以下特点：

- 开源项目已有成熟架构，不能随意重写；
- 数据库适配涉及连接驱动、SQL 语法、元数据查询、审核规则、执行链路等多个模块；
- 达梦数据库与 MySQL、Oracle 等数据库在语法、驱动、元数据结构上存在差异；
- 需要在理解原项目扩展机制的基础上进行最小侵入式改造；
- 一旦改造方式不当，容易破坏原有数据库类型的兼容性。

3.2 实践思路，采用“两阶段 AI 协作”的方式推进：

1 GPT：需求理解与任务提示词设计

2 ↓

3 Claude Code Plan Mode：基于真实仓库进行开发计划与代码实现

4

5 先与 GPT 沟通需求背景、技术目标、改造边界和风险点，由 GPT 帮助生成面向 Coding Agent 的高质量任务提示词；再将该提示词交给 Claude Code 的 Plan 模式，让其在 Archery 代码仓库中完成代码阅读、实现计划、分步开发和验证。

3.3 为什么要先用 GPT 生成任务提示词

在复杂开源项目二次开发中，直接让 Coding Agent “支持达梦数据库” 风险较高，因为这个指令过于宽泛，容易导致以下问题：

- Agent 不知道应该先阅读哪些模块；
- 可能跳过架构理解，直接新增代码；
- 容易误改原有 MySQL、Oracle、PostgreSQL 等数据库逻辑；
- 可能只完成表面适配，忽略元数据、语法差异和测试验证；

- 任务边界不清，导致实现范围失控。

因此，GPT 在这里承担的是 **需求分析师 + 架构助理 + Prompt Engineer** 的角色。

PS：这一步不也交给claude去做是因为GLM-5在处理类似需求任务时，输出内容质量不如GPT5.4（个人感受）

3.4 把“代码适配完成”变成“数据库行为验证通过”

SQL审核平台接入达梦测试脚本

提示词示例：



Archery-新增DM引擎核心代码：



 dm.py

当开发完成前后端功能测试完成后，如果只看代码，可能会误以为支持达梦数据库已经完成。但 SQL 审核平台真正的验收标准是：用户提交的达梦 SQL 能不能被正确审核、执行、返回结果，并且不破坏原有数据库类型。

PS：AI Coding 后不要只验功能“有没有实现”，而要验“业务行为是否成立”；用真实输入、边界场景、异常路径和回归验证，把代码结果转化为可交付结果。

场景四 跨系统交互链路重构类

示例：12345 与飞书交互链路的核心功能重构

优化前代码示例：

```
1
2 @Override
3 @Async
4 public void closeWorkOrder(CardFinishConfirmDTO cardFinishConfirmDTO,
    IssueTypeEnum issueTypeEnum) {
5     try {
6         OpsUserGroupInfoDTO opsUserGroupInfoDTO =
            opsUserGroupInfoService.findByWorkOrderNo(cardFinishConfirmDTO.getWorkOrderNo())
    );
7         if (ObjectUtil.isEmpty(opsUserGroupInfoDTO)) {
```



```

8         log.error("根据工单号{}未找到相关的用户组信息",
cardFinishConfirmDTO.getWorkOrderNo());
9         return;
10    }
11    AppTable[] appTables =
getAllAppTables(opsUserGroupInfoDTO.getSysCode());
12    AppTable handleTable = Arrays.stream(appTables).filter(
13        appTable ->
appTable.getName().contains(issueTypeEnum.getDescription())
14        ).findFirst().orElse(new AppTable());
15    // 创建请求体
16    // 创建请求对象
17    Map<String, Object> closeParams = new HashMap<>();
18    closeParams.put("工单状态", "已完成");
19    if
(StringUtils.isNotBlank(cardFinishConfirmDTO.getUserEvaluateInput())) {
20        closeParams.put("用户评价",
cardFinishConfirmDTO.getUserEvaluateInput());
21    }
22    if
(StringUtils.isNotBlank(cardFinishConfirmDTO.getUserEvaluateScore())) {
23        closeParams.put("评价得分",
cardFinishConfirmDTO.getUserEvaluateScore());
24    }
25    String messageViewUrl = "";
26    if (StringUtils.isNotBlank(feishuConfig.getMessage_view_url())) {
27        JSONObject jsonObject = new JSONObject();
28        messageViewUrl = String.format(feishuConfig.getMessage_view_url(),
opsUserGroupInfoDTO.getSysCode(), cardFinishConfirmDTO.getWorkOrderNo());
29        jsonObject.put("link", messageViewUrl);
30        jsonObject.put("text", "查看会话记录");
31        closeParams.put("会话记录", jsonObject);
32    }
33
34    Boolean hasUserSentMessage =
opsUserWorkOrdersService.hasUserSentMessage(cardFinishConfirmDTO.getWorkOrderNo
(), opsUserGroupInfoDTO.getUserLdap());
35    if (!hasUserSentMessage) {
36        closeParams.put("工单类型", "无效工单");
37    }
38    List<FeishuUserDTO> feishuUserDTOS = new ArrayList<>();
39    if (StringUtils.isNotBlank(opsUserGroupInfoDTO.getStuffLdaps())) {
40        //进行工单关闭, 查询服务的客服账号
41        //查找发送消息的客服
42        List<String> senderStuffLdap =
opsUserGroupMessageService.findStuffByWorkOrderNo(cardFinishConfirmDTO.getWorkO
rderNo());

```

```

43         //查找当前的客服
44         Set<String> ldapList =
changeStuffToList(opsUserGroupInfoDTO.getStuffLdaps().toLowerCase());
45         if (!CollectionUtils.isEmpty(senderStuffLdap)) {
46             for (String s : senderStuffLdap) {
47                 if (ldapList.contains(s.toLowerCase()) &&
!s.equalsIgnoreCase(opsUserGroupInfoDTO.getUserLdap())) {
48                     FeishuUserDTO feishuUserDTO = getFeishuUser(s);
49                     feishuUserDTOS.add(feishuUserDTO);
50                 }
51             }
52         }
53         //添加工单关闭进行回复的客服人员
54         if (!CollectionUtils.isEmpty(feishuUserDTOS)) {
55             closeParams.put("客服人员", createPersonFields(feishuUserDTOS));
56         }
57     }
58     //判断当前订单是否完成, 因为完成状态下, 用户还可以点击‘订单完成’卡片
59     Boolean isFinished =
opsUserWorkOrdersService.queryWorkOrderFinishStatus(cardFinishConfirmDTO.getWorkOrderNo());
60     if (!isFinished) {
61         closeParams.put("结单时间", new Date().getTime());
62     }
63     log.info("关闭工单参数, params={}", JSON.toJSONString(closeParams));
64     UpdateAppTableRecordReq req = UpdateAppTableRecordReq.newBuilder()
65
        .appToken(getFeishuConfig(opsUserGroupInfoDTO.getSysCode()).getBi_table_token())
66         .userIdType(GetUserUserIdTypeEnum.USER_ID.getValue())
67         .tableId(handleTable.getTableId())
68         .recordId(cardFinishConfirmDTO.getRecordId())
69         .ignoreConsistencyCheck(true)
70         .appTableRecord(AppTableRecord.newBuilder()
71             .fields(closeParams)
72             .build())
73         .build();
74     // 发起请求
75     UpdateAppTableRecordResp resp =
getFeiShuClient().bitable().v1().appTableRecord().update(req);
76     log.info("关闭工单结果, resp={}", JSON.toJSONString(resp));
77     exceptionHandle(resp);
78     opsUserGroupInfoService.closeWorkOrder(cardFinishConfirmDTO);
79
opsUserWorkOrdersService.closeWorkOrderNo(opsUserGroupInfoDTO.getLastWorkNo(),
feishuUserDTOS, messageViewUrl, cardFinishConfirmDTO, isFinished);
80     opsUserGroupMessageService.doAnalysisMessage(opsUserGroupInfoDTO);

```

```

81
82 //如果是未完全完成, 则进行AI分析
83 if (!isFinished) {
84     AIAalyzeDTO aiAnalyzeDTO = new AIAalyzeDTO();
85
86     aiAnalyzeDTO.setWorkOrderNumber(cardFinishConfirmDTO.getWorkOrderNo());
87     aiAnalyzeDTO.setSysCode(opsUserGroupInfoDTO.getSysCode());
88     aiAnalyzeDTO.setRecordId(cardFinishConfirmDTO.getRecordId());
89     //查看是否打开自定义类型
90     Boolean openCustomType =
91     sysBitableConfService.findOpenCustomTypeAnalysisSysCode(opsUserGroupInfoDTO.get
92     SysCode());
93     if (openCustomType) {
94         List<OpsSysDictV0> opsSysDictVOS =
95         opsSysDictService.getDictByCategory(opsUserGroupInfoDTO.getSysCode() +
96         DictConstants.CUSTOM_QUESTION_TYPE_SUFFIX);
97         if (!CollectionUtils.isEmpty(opsSysDictVOS)) {
98             aiAnalyzeDTO.setDoCustomSearch(true);
99             String questionTypeContext = opsSysDictVOS.stream()
100             .map(OpsSysDictV0::getDictLabel)
101             .collect(Collectors.joining(","));
102             aiAnalyzeDTO.setQuestionTypeContext(questionTypeContext);
103         }
104     }
105     List<AIToolServiceImpl.QAPair> qaPairs =
106     aiToolService.getAIAnswerByWorkOrderNum(aiAnalyzeDTO);
107     if (CollectionUtil.isEmpty(qaPairs)) {
108         AIToolServiceImpl.QAPair qaPair = qaPairs.get(0);
109         Map<String, Object> updateAIcontent = new HashMap<>();
110         updateAIcontent.put("工单类型", qaPair.getTicketType());
111         updateAIcontent.put("问题类型", qaPair.getQuestionType());
112         updateAIcontent.put("问题总结", qaPair.getIssueSummary());
113         updateAIcontent.put("回答总结", qaPair.getResponseSummary());
114         UpdateAppTableRecordReq aiReq =
115         UpdateAppTableRecordReq.newBuilder()
116         .appToken(getFeishuConfig(opsUserGroupInfoDTO.getSysCode()).getBi_table_token()
117         )
118         .userIdType(GetUserUserIdTypeEnum.USER_ID.getValue())
119         .tableId(handleTable.getTableId())
120         .recordId(cardFinishConfirmDTO.getRecordId())
121         .ignoreConsistencyCheck(true)
122         .appTableRecord(AppTableRecord.newBuilder()
123             .fields(updateAIcontent)
124             .build())
125         .build();
126     }
127     // 发起请求

```

```

119         UpdateAppTableRecordResp aiResp =
getFeiShuClient().bitable().v1().appTableRecord().update(aiReq);
120         log.info("closeWorkOrder更新工单AI内容结果, resp={}",
JSON.toJSONString(aiResp));
121         exceptionHandle(aiResp);
122
123         // 创建发送服务工单的卡片对象, 发送卡片
124         String contentResult = wrapperServerOrderConfirmCard(qaPair,
opsUserGroupInfoDTO);
125         CreateMessageReq req1 = CreateMessageReq.newBuilder()
126             .receiveIdType("chat_id")
127             .createMessageReqBody(CreateMessageReqBody.newBuilder()
128                 .receiveId(opsUserGroupInfoDTO.getChatId())
129                 .msgType("interactive")
130                 .content(contentResult)
131                 .build())
132             .build();
133         CreateMessageResp resp1 =
getFeiShuClient().im().v1().message().create(req1);
134         exceptionHandle(resp1);
135     }
136 }
137
138 } catch (Exception e) {
139
140     log.error("关闭工单异常, recordId: {}, issueType: {}",
cardFinishConfirmDTO.getRecordId(), issueTypeEnum, e);
141 }
142
143
144 }

```

优化后

```

1 @Override
2 @Async("orderStatusTaskExecutor")
3 public void closeWorkOrder(CardFinishConfirmDTO cardFinishConfirmDTO,
IssueTypeEnum issueTypeEnum) {
4     try {
5         // 1. 查询基础信息
6         OpsUserGroupInfoDTO groupInfo =
getGroupInfo(cardFinishConfirmDTO.getWorkOrderNo());
7         if (ObjectUtil.isEmpty(groupInfo)) {
8             log.error("根据工单号{}未找到相关的用户组信息",
cardFinishConfirmDTO.getWorkOrderNo());

```

```

9         return;
10    }
11
12    // 2. 获取多维表格信息
13    AppTable handleTable = getHandleTable(groupInfo.getSysCode(),
14        issueTypeEnum);
15
16    // 3. 构建关闭参数
17    Map<String, Object> closeParams =
18        buildCloseParams(cardFinishConfirmDTO, groupInfo);
19
20    // 4. 更新工单状态到飞书
21    updateWorkOrderToFeishu(groupInfo, handleTable,
22        cardFinishConfirmDTO.getRecordId(), closeParams);
23
24    // 5. 关闭本地工单状态
25    closeLocalWorkOrder(cardFinishConfirmDTO, groupInfo, closeParams);
26
27    // 6. 问答对AI分析（如果需要）
28    performAIAnalysis(cardFinishConfirmDTO, groupInfo, handleTable);
29
30    } catch (Exception e) {
31        log.error("关闭工单异常, recordId: {}, issueType: {}",
32            cardFinishConfirmDTO.getRecordId(), issueTypeEnum, e);
33    }
34 }

```

对比

之前：旧实现是一个约 140 行的方法中，查数据、组字段、调飞书、工单状态、dify AI 工作流、发送飞书卡片 全揉在一起 FeishuApiServiceImpl.java (line 1619)

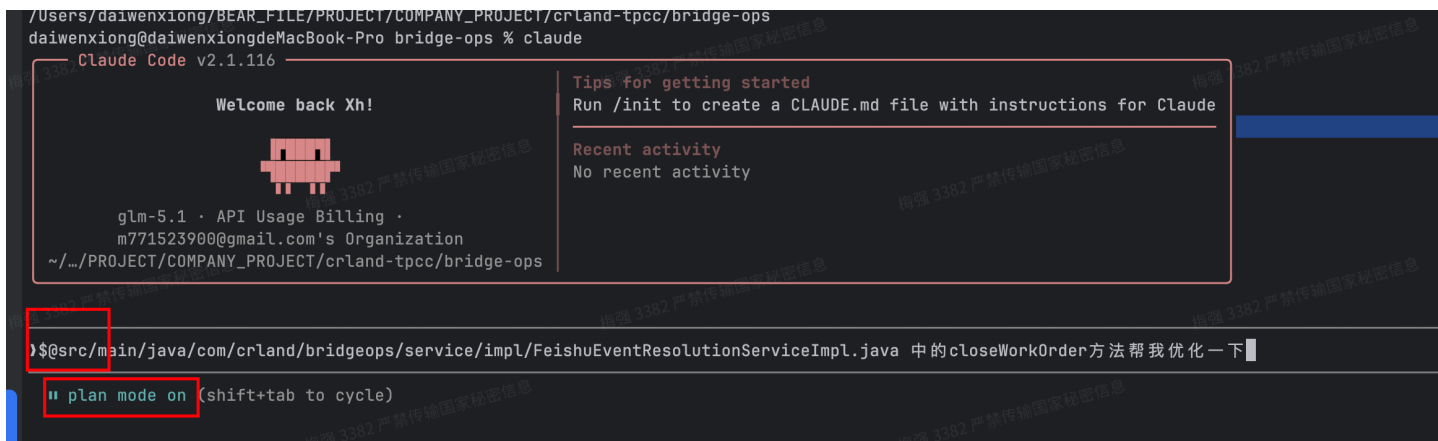
之后：主方法缩到约 28 行，方法名本身就能说明流程，阅读和排查明显更快。
FeishuApiServiceImpl.java (line 1348)

之前：字段名全靠硬编码字符串维护，重复且脆弱。

之后：字段常量集中管理，可维护性更强。

之前：改某个局部逻辑时很容易牵一发而动全身。

之后：每个步骤都有独立入口，后续补测试、继续拆分类都会更顺利。



指令：

- 1 @src/main/java/com/crland/bridgeops/service/impl/FeishuEventResolutionServiceImpl.java 中的closeWorkOrder方法帮我优化一下
- 2 --
- 3 @ 指令的用于**收敛上下文**。让模型只关注这次关单流程相关的代码，而不是在整个仓库里发散分析。这样可以减少无关代码干扰，避免 AI 顺手改到不该动的模块，也能让重构目标更聚焦，比如明确就是优化 closeWorkOrder 这一段逻辑，而不是泛化成一次大范围改造。
- 4
- 5 plan mode claude的计划模式是 **先设计、后实施**。在开始修改前，模型会先把当前问题拆成几个步骤，比如：哪些逻辑应该抽方法、哪些字段应该提常量、哪些流程应该从主方法里剥离出来，这样做能避免边改边想造成的结构混乱。

场景五 关于BUG 的 AI 辅助定位与修复

5.1示例一：跨系统/服务 调用链 BUG 修复

现象：项目管理系统调用猪齿鱼发版授权接口失败导致用户授权失败

问题点：

1. 代码分散：调用方和被调用方属于两个不同项目（微服务同理）
2. 链路复杂：参数从页面 / 业务对象 / 配置 / 数据库状态中拼装
3. 运行成本高：不一定能完整启动两个系统，只能通过代码 + 数据库 + 日志推理
4. 需要多人协作排查

核心操作：

1. 先对两个项目分别 `/init` ，再把两个项目代码放到同一个目录下。

目的：把两个独立项目变成“统一排查上下文”

- 1 由于调用方和被调用方分属两个代码仓库，单独分析任意一个项目都无法完整理解问题。我先**分别初始**
化两个项目上下文，再将两个项目放到同一工作空间，使 AI 能同时读取项目管理系统和猪齿鱼接口服
务的代码，从而具备跨仓库分析调用链的能力。

**跨系统/服务 BUG 排查时，要先构造统一上下文空间，让 AI 同时理解调用方、被调用方、接口契约和
数据状态。**

2. 先让coding agent分析调用链模型，以及关键类、关键方法、作用描述。

目的：先不急着“报错点排查”优先“调用链还原”，避免上下文腐化（梳理调用链本身就会占用上
下文，可能会加载一些无用类文件，所以调用链梳理完后，再新开一个会话，把本次输出内容当作输
入。）

结果示例如下：

环节	关键类 / 方法	作用
评审入 口	OnlineReviewController.addSubmitReview (line 164)、 updateSubmitReview	提交/修改上线评审
评审分 发	ReviewInfoServiceImpl.addReview (line 672)、 updateReview	判断是“上线评审”后转给 ReviewOnlineService
发版环 境落库	ReviewOnlineServiceImpl.addReview (line 122)、 updateReview	保存评审单，同时保存发版环境、发 版人
发版环 境/人员 明细	ReviewOnlineServiceImpl.addReviewEnvList (line 433)、addReviewEnvUserList	写入 t_review_env、 t_review_env_user；发版人来自环 境版本人员表
本地同 步 PSP	LocalSynPspJob.localSynPspUserUdoPermission (line 108)	扫描到达发版开始/结束时间的环境， 触发授权或回收
.....		
猪齿鱼 环境成 员更新	C7nManager.updateEnvMembers (line 67)	先查猪齿鱼环境当前用户列表，再合 并新增/移除后的完整用户列表
最终调 用猪齿 鱼	ChoerodonServiceImpl.authOrRecycling (line 193)	POST /devops/v1/projects/{project_id}/en vs/{envId}/permission，body 是最 终环境用户 ID 列表

梳理调用链，有助于排查失败到底发生在哪个阶段，调用前参数组装、HTTP 调用、被调方入参校验、权限判断，还是数据库状态不满足等（如果开发本身有判断的话，可以指定coding agent排查某个链路）

3. 让 AI 结合数据库状态推理，而不是只看静态代码。

目的：代码逻辑 + 真实数据状态的联合排查。（coding agent 自带命令行工具，可以使用一些客户端工具直连数据库，本例中是msysql client）

- 1 跨服务调用失败很多时候不是代码写错，而是数据状态不符合预期，例如：
- 2 项目编码不一致；
- 3 应用 ID 映射缺失；
- 4 用户没有绑定猪齿鱼账号；
- 5 角色授权关系不存在；

对跨系统调用类 BUG，静态代码只能解释“应该怎么走”，数据库状态才能证明“实际为什么失败”。因此，本案例让 AI 根据调用链自动查询测试库，逐步验证每个关键业务条件是否成立。

4.项目不用跑起来，也能完成高质量定位

之前：必须启动服务、打断点、联调

coding agent：在服务无法快速启动或联调成本较高时，可以尝试通过“代码静态分析 + 数据库查询 + 调用链推理”的方式完成问题定位。

本次排查过程中，并没有强依赖完整启动两个系统，而是让 AI 通过阅读代码、追踪方法调用、分析参数来源，并结合测试环境数据库查询，模拟接口调用过程中的关键判断路径，最终定位失败原因。

5.2示例二： SRE Copilot 风险告警卡片BUG修复



T json_template.txt

现象：

风险响应告警卡片在特定告警类型下渲染异常，部分字段不展示，按钮点击后无法正确回调。经排查发现，卡片 JSON 拼接过程中部分字段结构不符合飞书卡片 schema，且动态变量在某些场景下为空，导致卡片组件渲染失败或交互异常

问题点：

- JSON 层级深，字段多；
- 一个字段写错，整张卡片可能渲染异常；
- 拼接逻辑分散在多个函数或模板中；
- 卡片内容既有业务数据，又有展示协议；
- 错误往往不是语法错误，而是结构不符合飞书卡片协议；
- 人工靠肉眼排查容易遗漏括号、数组层级、组件字段、变量占位符。

核心输入：

- 1.异常卡片截图 / 报错信息
- 2.触发该卡片的风险告警样例数据
- 3.项目中卡片 JSON 拼接代码

4.飞书卡片官方开发文档 (<https://open.feishu.cn/document/feishu-cards/card-json-v2-structure>)

5.期望展示效果

通过补充飞书卡片开发文档，AI 能够将卡片 JSON 与官方协议进行对照，判断字段是否缺失、组件层级是否错误、交互参数是否符合规范，从而提升问题定位的准确性。

PS：外部平台类 BUG 要让 AI 同时理解“项目代码”和“平台协议”；只有把官方文档作为排查依据，AI 才能从猜测式修复转向规则化定位。

5.3示例三： 代建报表查询 SQL BUG



现象：

部分业务指标统计结果与实际数据不一致。该报表依赖一段超长SQL，包含多个子查询、多表关联、聚合统计和条件过滤。人工直接阅读 SQL 很难快速定位问题。通过 AI 辅助，将 SQL 拆解为数据来源、关联关系、过滤条件、聚合口径和输出字段，最终定位到某个过滤条件或 join 逻辑导致部分数据被漏算。

问题点：

- 某个统计值不对；
- 某类数据漏算；

- 某个状态被重复统计；
- 字段口径和业务口径不一致；
- 权限或组织过滤条件缺失；
- null 值处理导致统计偏差。

.....

核心输入：

1.现象

2.查询条件、时间范围、组织范围

3.核心对象：问题SQL

4.要求输出内容：

- 1 输出：报表结果
- 2 根因：过滤条件 / join 条件 / 聚合口径错误
- 3 修复：调整 SQL 条件或拆分逻辑
- 4 验证：用样例数据或对账 SQL 验证指标

复杂 SQL 类 BUG，不要让 AI 直接重写 SQL，而要先让 AI 拆解 SQL 结构、标注数据流、解释指标口径，再基于实际异常指标定位最小修改点。

PS:超长 SQL 最大风险不是“写不出来”，而是“改错了还不知道影响了谁”。AI 修复 SQL 时必须要求它输出影响范围、变更前后口径差异、验证 SQL 和回归检查点。

相关配置及技巧

控制活跃工具数量：

- 尽管可以配置很多MCP（模型上下文协议）服务器和插件，但应**严格控制同时启用的数量**（建议活跃MCP少于10个，总活跃工具少于80个），以保障上下文窗口性能和减少Token消耗。

战略性上下文压缩：避免自动压缩，改为在逻辑节点（如完成一个探索阶段或里程碑后）**手动触发压缩**（ /compact ），以更好地保留相关上下文

其他技巧

@文件 限制范围

！ 执行命令 无需退出CLI